

Stochastic Local Search Algorithms for the Linear Ordering Problem

INFO-H-413 — Heuristic Optimization — Implementation Exercise 2

Université Libre de Bruxelles — 2026

Luis Brunard (577721)

Abstract

We design and evaluate two stochastic local search algorithms for the Linear Ordering Problem: Simulated Annealing (SA) and a MAX-MIN Ant System (MMAS). Both build on the VND local search from the first implementation exercise. SA explores the search space through random insert moves with a temperature-controlled acceptance criterion. MMAS constructs solutions using pheromone trails and heuristic information, then refines each one with VND. Both algorithms are evaluated on all instances of size 150 and compared using Wilcoxon signed-rank tests and run-time distributions.

1. Introduction

In the first implementation exercise, we studied iterative improvement algorithms for the Linear Ordering Problem (LOP). We found that Insert with first-improvement and CW initialisation gives the best single-neighbourhood results, and that VND-TIE slightly improves on it. But all these methods stop at the first local optimum they reach. There is no mechanism to escape and keep searching.

This exercise addresses that limitation. We implement two stochastic local search (SLS) algorithms that can escape local optima and continue exploring the search space over a longer time budget. The two methods come from different classes: Simulated Annealing is a simple SLS method, and MAX-MIN Ant System is a population-based method. Both use VND from the first exercise as a building block.

The question we are trying to answer is whether these more sophisticated methods can find significantly better solutions than plain local search, and how they compare to each other.

2. Algorithms

2.1. Simulated Annealing

Simulated Annealing (SA) is inspired by the annealing process in metallurgy [1]. The idea is to allow the search to accept worsening moves with some probability, so that it can escape local optima. That probability decreases over time: early on the algorithm explores freely, later it becomes more selective and converges.

2.1.1. Initial solution

We start from a CW solution (Chenery–Watanabe heuristic, as described in the first exercise). CW ranks elements by their row-sum score and gives a reasonable starting point without any local search overhead. We do not apply VND before SA starts, so the full time budget is available for the SA search itself.

2.1.2. Neighbourhood and move generation

We use the **insert neighbourhood**: pick a random element at position i and reinsert it at a random position $j \neq i$. This was the strongest neighbourhood in the first exercise and it gives SA enough freedom to rearrange the permutation significantly in a single step. At each iteration, one random insert move is sampled. The gain Δ is computed using the $O(n)$ delta evaluation from the first exercise.

2.1.3. Acceptance criterion

If the move improves the solution ($\Delta > 0$), it is always accepted. If it worsens it ($\Delta < 0$), it is accepted with probability:

$$P(\text{accept}) = \exp\left(\frac{\Delta}{T}\right)$$

where T is the current temperature. When T is high, the exponential is close to 1 and almost everything is accepted. When T is low, only small degradations have a chance.

2.1.4. Temperature schedule

The initial temperature T_0 is calibrated automatically. Before starting the main loop, we sample 500 random insert moves and compute the average absolute value of the worsening deltas. We set T_0 so that roughly 50% of those moves would be accepted:

$$T_0 = \frac{|\overline{\Delta^-}|}{\ln(2)}$$

This avoids having to guess a problem-specific temperature. The cooling follows a geometric schedule: after each temperature level, we multiply T by a cooling rate $\alpha = 0.99$. Each temperature level consists of $L = n \times 10$ random moves, where n is the instance size. This gives a slow enough cooling that the algorithm has time to explore at each temperature.

2.1.5. Final refinement

When the time limit is reached, we take the best solution found during the entire SA run and apply VND-TEI to it. This ensures that the final solution is at least a local optimum with respect to all three neighbourhoods.

2.1.6. Summary

- **Initialisation:** CW heuristic
- **Neighbourhood:** Insert (random move at each step)
- **Acceptance:** Metropolis criterion with Boltzmann probability
- **Cooling:** Geometric ($\alpha = 0.99$), $L = 10n$ moves per level
- **Calibration:** T_0 set from average worsening delta (50% acceptance)
- **Post-processing:** VND on the best solution found

2.2. MAX–MIN Ant System

Ant Colony Optimisation (ACO) is a population-based metaheuristic inspired by the foraging behaviour of ants [2]. A colony of artificial ants constructs solutions probabilistically, guided by pheromone trails that encode information from previous good solutions. We use the MAX–MIN Ant System (MMAS) variant [3], which bounds the pheromone values to avoid premature convergence.

2.2.1. Pheromone representation

We maintain a pheromone matrix $\tau \in \mathbb{R}^{n \times n}$, where τ_{ij} represents the learned desirability of placing element i before element j in the permutation. All entries are initialised to $\tau_{\max}$ at the start, which encourages exploration in the first iterations.

2.2.2. Construction phase

Each ant builds a complete permutation by placing elements one at a time. At each step, the ant has a set R of remaining (unplaced) elements. For each candidate $j \in R$, we compute:

- A **pheromone score**: $\varphi(j) = \sum_{k \in R, k \neq j} \tau_{jk}$, which measures how much past experience favours placing j before the other remaining elements.
- A **heuristic score**: $\eta(j) = \sum_{k \in R, k \neq j} c_{jk}$, which is the direct benefit of placing j before the remaining elements according to the cost matrix.

The probability of choosing element j is then:

$$p(j) = \frac{[\varphi(j)]^\alpha \cdot [\eta(j)]^\beta}{\sum_{l \in R} [\varphi(l)]^\alpha \cdot [\eta(l)]^\beta}$$

where α and β control the relative importance of pheromone versus heuristic information. We use $\alpha = 1$ and $\beta = 3$, giving more weight to the problem-specific heuristic while still letting pheromone guide the search toward previously successful regions.

2.2.3. Local search

After construction, each ant’s solution is improved using **VND-TEI** (Transpose $\rightarrow$ Exchange $\rightarrow$ Insert, first-improvement). This is the most important step: the constructed solution is typically far from a local optimum, and VND brings it to a much better quality. Without this step, ACO would not be competitive with SA.

2.2.4. Pheromone update

After all ants have constructed and improved their solutions, the pheromone matrix is updated in two phases:

Evaporation. All pheromone values are reduced by a factor $(1 - \rho)$:

$$\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij}$$

This gradually forgets old information and prevents the accumulation of pheromone on suboptimal edges. We use $\rho = 0.2$.

Deposit. The best-so-far solution reinforces the pheromone on its edges. For every pair of positions (p, q) with $p < q$ in the best permutation:

$$\tau_{s_p^*, s_q^*} \leftarrow \tau_{s_p^*, s_q^*} + 1$$

Using the best-so-far solution (rather than the iteration-best) provides a stronger learning signal and pushes the colony toward the best known region of the search space.

Bounds. After each update, all pheromone values are clamped to $[\tau_{\min}, \tau_{\max}]$. This is the core mechanism of MMAS: it prevents any trail from becoming so dominant that the construction becomes deterministic, and it prevents unused trails from vanishing completely.

2.2.5. Parameters

Parameter	Value	Role
m (ants)	10	Number of solutions constructed per iteration
α	1.0	Pheromone weight in construction probability
β	3.0	Heuristic weight in construction probability
ρ	0.2	Evaporation rate
$\tau_{\max}$	10.0	Upper bound on pheromone values
$\tau_{\min}$	0.1	Lower bound on pheromone values

Table 1: ACO parameter settings.

The number of ants $m = 10$ is a compromise between diversity (more ants explore more of the search space per iteration) and cost (each ant requires a full VND run). With $n = 150$, each VND call takes a non-negligible amount of time, so using too many ants would leave too few iterations for pheromone learning.

The choice $\beta = 3$ gives a strong bias toward the heuristic information. This makes sense because the CW-style heuristic (sum of row weights) is already a good predictor of element quality. The pheromone then fine-tunes the ordering based on experience.

2.2.6. Summary

- **Construction:** Probabilistic, guided by pheromone (τ) and heuristic (η)
- **Local search:** VND-TEI applied to every constructed solution
- **Pheromone update:** Evaporation ($\rho = 0.2$) + deposit on best-so-far
- **Bounds:** MMAS with $\tau \in [0.1, 10.0]$
- **Colony size:** 10 ants per iteration

3. Experimental Setup

3.1. Instances and termination criterion

Both algorithms were run once on each instance of size 150. The termination criterion is defined as:

$$t_{\max} = \overline{t_{\text{VND}}} \times 500$$

where $\overline{t_{\text{VND}}}$ is the average computation time of a full VND run on instances of the same size, re-measured in the experimental environment used for this exercise (**Average VND time : 0.733433s**). This yields a time budget of **Time limit (x500): 366.72s** per run, long enough for the SLS algorithms to perform a meaningful search.

3.2. Implementation

The solver is written in C, compiled with GCC and -O3. Because the making all experimental would have taken several days on a laptop, all measurements were made using a dedicated DigitalOcean Droplet (2 vCPU, Ubuntu 24.04) using a tmux session for persistence. Both SA and ACO reuse the VND, delta evaluation functions and instance reader from the first exercise. The random number generator is the same pseudo-random generator used in the entire project. Seeds can be set via the --seed command-line flag to allow reproducible runs.

3.3. Evaluation metric

We use the same **Relative Percentage Deviation** (RPD) as in the first exercise:

$$\text{RPD} = \frac{\text{BestKnown} - \text{FinalCost}}{\text{BestKnown}} \times 100$$

Lower is better. An RPD of 0 means we matched the best-known solution.

4. Exercise 2.1 — SLS Results on Size 150 Instances

Table 2 reports the main results for both SLS algorithms across all 39 instances of size 150, together with the best VND configuration (VND-TIE) from Exercise 1 used as a baseline.

Algorithm	Avg RPD	SD	Min RPD	Max RPD	Avg time (s)
Simulated Annealing	0.435	0.160	0.152	0.817	366.72
ACO (MMAS)	0.910	0.193	0.556	1.405	370.62
VND-TIE (Ex. 1)	1.690	0.485	0.828	2.747	0.41

Table 2: Average RPD (%) over all 39 size-150 instances for each SLS algorithm and the best VND baseline from Exercise 1.

Figure 1 summarise the same information graphically.

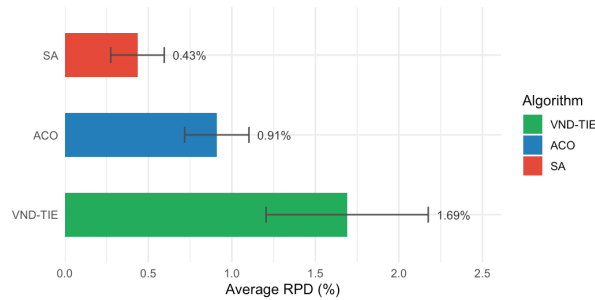


Figure 1: Average RPD (%) for the two SLS algorithms and the VND-TIE baseline from Exercise 1. Error bars represent one standard deviation.

4.1. Correlation Plot

Figure 2 plots, for each of the 39 instances, the SA RPD on the x -axis against the ACO RPD on the y -axis.

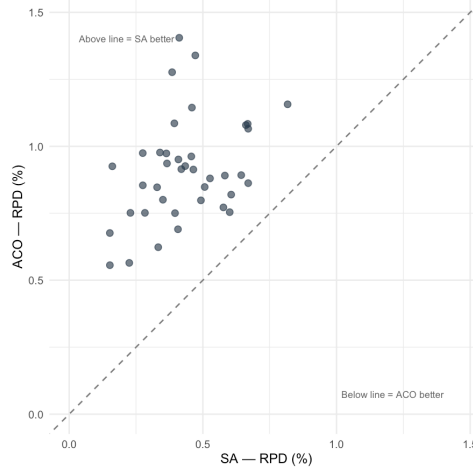


Figure 2: Correlation plot of RPD values: SA (x -axis) vs ACO (y -axis). Each point is one size-150 instance. The dashed line is $y = x$. Points below the line favour ACO; points above favour SA; points on the line same result for two SLS.

Figure 3 gives a per-instance view, sorted by the RPD obtained by SA.

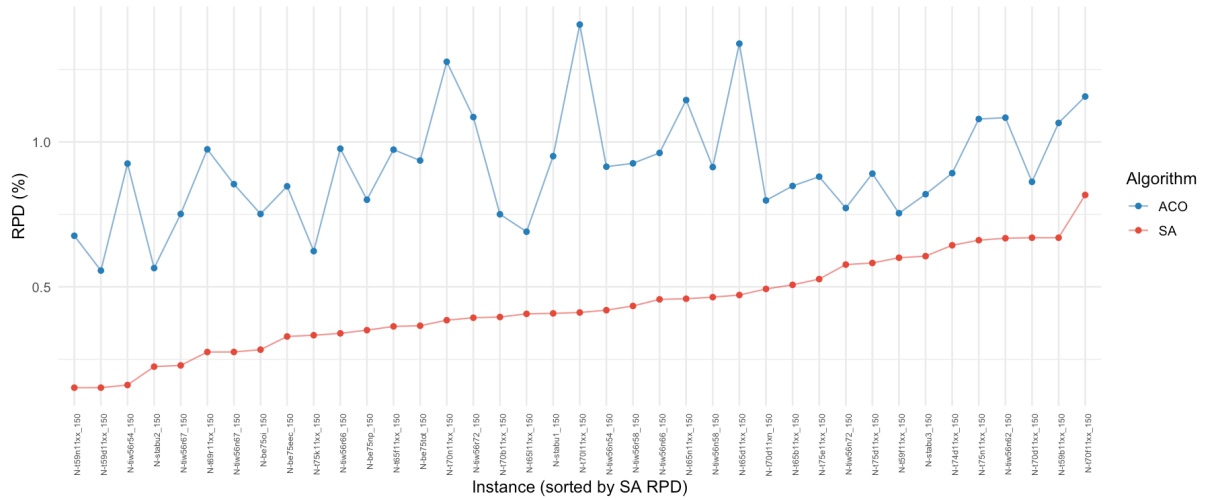


Figure 3: Per-instance RPD (%) for SA and ACO. Instances are ordered by increasing SA RPD.

4.2. Observations

SA has the lowest average RPD at 0.435% (SD = 0.160), about half of what ACO achieves (0.910%, SD = 0.193). Both are well below the VND-TIE baseline from Exercise 1 (1.690%), which was expected since VND stops at the first local optimum after less than half a second, while the SLS methods search for over six minutes.

What is more interesting is how consistent the gap is. In Figure 2, every single point lies above the diagonal: SA beats ACO on all 39 instances, not just on average. Figure 3 confirms this – the SA curve stays below the ACO curve everywhere. The difference between the two is not driven by a few easy or hard instances; it is a systematic effect across the whole benchmark.

4.3. Statistical Tests

We apply the Wilcoxon signed-rank test at significance level $\alpha = 0.05$ to determine whether the differences observed above are statistically significant [4].

Algorithm A	Algorithm B	p-value	Significant?
SA	ACO (MMAS)	$< 10^{-7}$	Yes
SA	VND-TIE (Ex. 1)	$< 10^{-7}$	Yes
ACO (MMAS)	VND-TIE (Ex. 1)	$< 10^{-7}$	Yes

Table 3: Wilcoxon signed-rank test results ($\alpha = 0.05$) comparing the SLS algorithms and the best configuration from Exercise 1. Each test is paired over the 39 instances of size 150.

All three differences are significant at $\alpha = 0.05$ with p -values numerically indistinguishable from zero (the Wilcoxon statistic $W = 0$ for both SA-vs-ACO and SA-vs-VND, meaning that on every paired instance the algorithm on the left was strictly better). We can therefore confidently rank the three approaches as $SA \prec ACO \prec VND\text{-}TIE$ with respect to RPD.

5. Run-Time Distributions

For each algorithm, we ran 25 independent repetitions on the first two instances of size 150 (N-be75eec_150 and N-be75np_150), with a cut-off time of $10 \times t_{\max} \approx 3667$ s.

Ideally, an RTD curve shows $P(\text{solve})$ as a function of elapsed time, which requires the solver to periodically log the quality of its current best solution during the run. Our implementation only records the final solution once the time budget is exhausted, so we cannot plot a meaningful $P(\text{solve})$ curve over time. What we can do is compare the *distribution of final solution qualities* across the 25 seeds and compute the fraction of runs that reached a given target at the end. This is equivalent to reading the rightmost point of the RTD curve and gives us the success rate at $t = t_{\text{cutoff}}$.

We consider three target levels: $RPD \leq 0.5\%$, $RPD \leq 0.25\%$, and $RPD \leq 0.1\%$ from the best-known solution.

Instance	Algo	$\leq 0.5\%$	$\leq 0.25\%$	$\leq 0.1\%$
N-be75eec_150	SA	60%	20%	0%
N-be75eec_150	ACO	12%	0%	0%
N-be75np_150	SA	64%	0%	0%
N-be75np_150	ACO	12%	0%	0%

Table 4: Fraction of the 25 runs reaching each quality target at the cut-off (≈ 3667 s).

Figure 4 shows the distribution of final RPD values across the 25 seeds for each combination of instance and algorithm. Figure 5 presents the success rates from Table 4 as a bar chart.

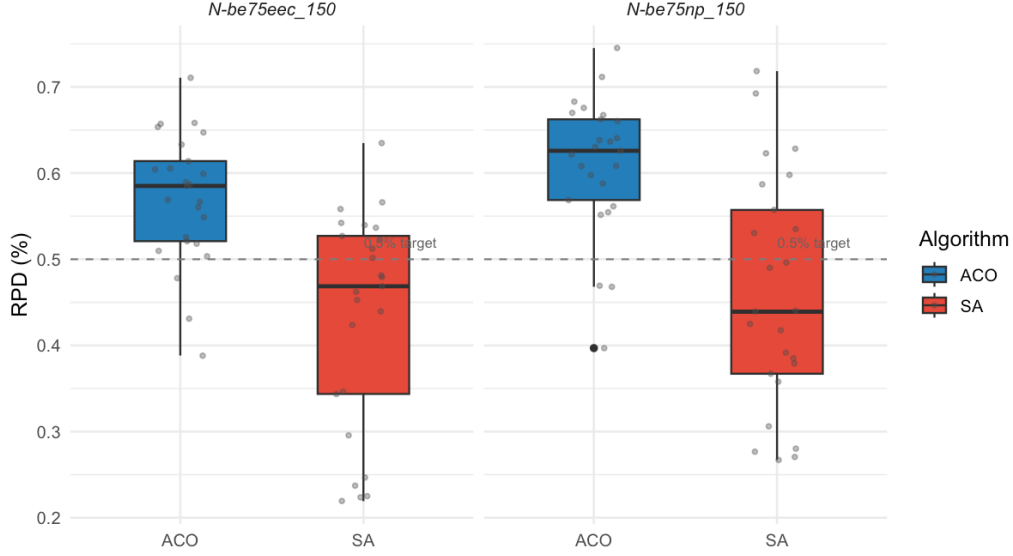


Figure 4: Distribution of final RPD (%) across 25 seeds for SA and ACO on the two RTD instances. Each grey dot is one run. The dashed line marks the 0.5% target.

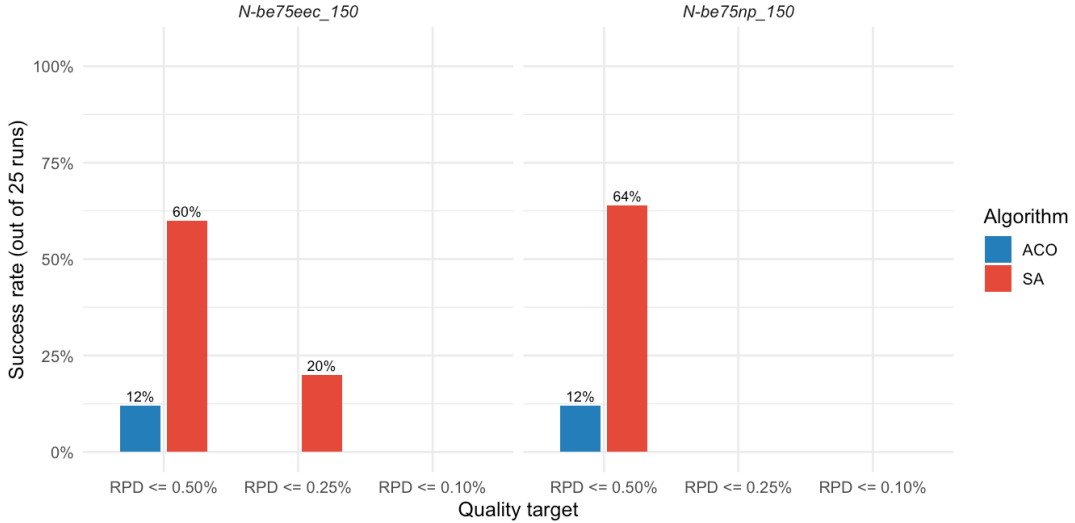


Figure 5: Empirical success rates at three quality thresholds (0.5%, 0.25%, 0.1% from best-known) over 25 runs.

5.1. Observations

The box plots in Figure 4 show a clear gap between the two algorithms. SA’s median RPD sits around 0.44–0.47% on both instances, with several runs going as low as 0.22%. ACO’s median is higher (0.58–0.62%) and the distribution is more concentrated above the 0.5% line. The two instances produce similar distributions, so this is not an instance-specific effect.

The success rate chart (Figure 5) puts numbers on this gap. At the 0.5% threshold, SA succeeds in 60–64% of the runs while ACO only reaches it 12% of the time. When we tighten the target to 0.25%, SA still manages 20% on *N-be75eec_150*, but ACO never gets there. At 0.1%, neither algorithm succeeds within the budget. This last point is interesting: even with nearly an hour of computation per run, getting within 0.1% of the best-known remains out of reach, which gives a sense of how hard these instances still are for these methods.

The wider spread of SA toward lower RPD values suggests that SA explores the search space more effectively than ACO in this time budget. ACO’s tighter distribution probably reflects the small number of pheromone-update cycles that fit within the budget, since each ant has to run a full VND before the colony can learn anything.

6. Conclusion

We implemented two SLS algorithms for the Linear Ordering Problem: Simulated Annealing (SA) with insert moves and automatic temperature calibration, and a MAX–MIN Ant System (MMAS) where each ant’s solution is refined by VND-TEI. Both reuse the VND from the first exercise and run for $t_{\max} \approx 367$ s per instance ($500 \times$ the average VND time).

On the 39 instances of size 150, SA reached an average RPD of 0.435%, ACO 0.910%, and VND-TIE from Exercise 1 sat at 1.690%. All pairwise differences are statistically significant (Wilcoxon, $p < 10^{-7}$). SA beat ACO on every single instance, not just on average. The 25-seed experiments on two instances showed that SA reaches the 0.5% target in about 60% of the runs, ACO only in 12%.

ACO’s weaker performance probably comes from the cost of running VND inside every ant. At $n = 150$, each VND call takes close to a second, so the colony only gets through a handful of construct-update cycles in the time budget. The pheromone matrix barely has time to converge on anything useful. SA does not have this problem: individual insert moves are cheap, and it can evaluate millions of them in the same time.

For this instance size and time budget, SA is the better choice. It is simpler to implement, has fewer parameters to tune (the temperature calibrates itself), and it produced the best results across the board.

References

- [1] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, *Optimization by Simulated Annealing*, vol. 220, no. 4598. 1983, pp. 671–680.
- [2] Wikipedia contributors, “Ant colony optimization algorithms.” [Online]. Available: https://en.wikipedia.org/wiki/Ant_colony_optimization_algorithms
- [3] T. Stützle and H. H. Hoos, “MAX–MIN Ant System,” *Future Generation Computer Systems*, vol. 16, no. 8, pp. 889–914, 2000.
- [4] Wikipedia contributors, “Wilcoxon signed-rank test.” [Online]. Available: https://en.wikipedia.org/wiki/Wilcoxon_signed-rank_test